

This is a copy of a shared ChatGPT conversation
Report conversation

📁 Uploaded a file

This is the output Cursor produced when I asked it to run Instruments in the way you specified. I'm not sure what, if anything I should be doing in response to this data. I don't really understand why some paths were designated WARN or what I can or should do about it.

I'll read the report and translate the Instruments findings into "what matters / what to ignore / what to ask Cursor to do next."

Stopped thinking >

This is actually a **good report**. Nothing here says "stop beta" or "fix immediately."

The **WARN** labels mostly mean:

1. Instruments saw tiny "microhangs."

Cursor says these were likely caused by Instruments attaching/profiling, not something users would notice. Since there are no repeated hangs, no console warnings, no CPU-heavy app code, and no janky pattern, I would not treat these as bugs yet.

2. Some flows look slow because the test harness waits.

Search/Add showing ~8.5s is mostly from `waitForSearchResults()` and network/debounce timing, not your app doing 8 seconds of work. The more useful number is TTFF around **1.3–1.5s**, though even that may be inflated under Instruments.

3. Duplicate search in Add to Wishlist is expected.

The report says the profile flow searches before adding, so that duplicate is probably test automation behavior, not necessarily app behavior.

What matters most:

- **0 leaks**
- **No retained view/view model instances**
- **Memory returns to idle**
- **No monotonic memory growth in stress loops**
- **No Swift concurrency/runtime warnings**
- **Cold launch around 1 second**

My recommendation: **do not make performance changes from this report yet**. Instead, ask Cursor for one follow-up:

Run the same core flows manually or via UI tests without Instruments attached, logging only app signposts/OSLog timings, and compare perceived user-facing durations against the Instruments results. Specifically confirm whether Search, Show Details, and Add to Wishlist feel slow in normal use, and whether any main-thread hangs reproduce outside Instruments.

For beta: **performance/resource usage looks safe**. Your bigger concern is still the occasional idle crash, which needs long-duration logging/crash capture rather than this flow-based profiling.

This is what Cursor reported. Does it seem complete to you?

Thought for a second >

Yes — for the question we asked, it seems complete.

It answers the key things:

1. Are the WARN paths actually slow in normal use?

No: Search, Show Details, and Add to Wishlist are all ~400–600 ms user-facing.

2. Were Instruments numbers misleading?

Yes: the 6–8 second numbers were mostly automation waits/profiler overhead.

3. Do main-thread hangs reproduce without Instruments?

No.

4. Can you rerun the test?

Yes: it added a script and saved reports.

The one thing I'd still want from Cursor is a **cleanup/commit recommendation**, because it modified code to support this. Ask:

Please list every source/script/project file changed for uninstrumented profiling, and classify each as keep permanently, DEBUG-only, or remove before beta/App Store. Confirm no profiling-only behavior can run in normal app launches.

But as a performance investigation: **yes, this is enough**. I would not chase the WARNs further right now.